Product Review: Custom Reporting

Reviewed document: Custom Reporting, Product Requirements Document (as provided).

This is a review of the spec the way I'd review it after development wraps, before signing off on shipping it. Three parts: a prioritized look at what's missing or unclear in the spec (A), a test plan for what needs verifying before release (B), and what I'd want on a dashboard once it's live (C). B leans on A directly: several test cases exist specifically to check whichever way an open question from Part A gets decided.

Part A: Product Review

The spec reads well at the product-narrative level. The problem statement lands, and the two report types map cleanly onto two real people: the exec who wants a PDF, the practitioner who wants a CSV to work from. Where it gets thin is in the operational details: what happens at the edges of normal usage, plus a handful of questions that matter more for Upwind than they might for most companies, given the product itself exists to protect exactly this kind of data.

I picked the five gaps below because each one changes what actually gets built, not just how it's worded. A few more are listed briefly afterward, real but lower stakes.

1. Who is a report actually scoped to, the creator or the viewer?

A report can be set to "Organization level" visibility, or emailed out to a list of people. Nowhere does the spec say whether the data *inside* it reflects the access of whoever created it, or gets re-filtered for whoever opens it. Both output formats are static files generated once (a rendered PDF, an exported CSV), so there isn't really a way to re-filter per viewer after the fact. That's probably the intended answer, but the spec never says so, and it's an easy thing to quietly assume RBAC "just handles."

It matters because the failure mode is silent. Someone with broad access builds a dashboard, turns it into an org-wide report, and now everyone on that distribution list, including people whose own role would never grant them that visibility directly, can see it. For most products that's an awkward permissions bug. For a company whose entire product is security posture data, it's exactly the kind of thing a customer's own security review would catch.

Closing it needs one explicit product decision, plus two follow-ups: warning copy in the UI when a report is set to org-wide visibility or a broad recipient list, and a decision on whether creating that kind of report should require its own permission rather than riding on general report-creation access. One likely model for the core decision is that content is generated according to the creator's access, but that needs to be stated explicitly rather than assumed.

2. Nothing stops a recipient from being outside the company

The workflow mockup shows recipients as plain email addresses, with no indication they have to belong to existing Upwind users. Pair that with gap #1: if content reflects the creator's full access, and any address can be added as a recipient, you've got a clean, repeatable way to route a full security-posture export to an outside inbox on a recurring schedule. That's not a hypothetical misuse case. It's just what the feature does as specified, if nobody restricts it.

I'd want this decided before release, not discovered after: recipients validated against the org's own user list by default, with external recipients allowed only behind a separate, logged permission if there's a real business need (sending a report to an outside auditor is legitimate; it just shouldn't be the default, unlogged path).

3. The "authenticated download link" for large files raises more questions than it answers

Straight from the FAQ: oversized reports get delivered as "a link that, upon authentication, downloads the report directly." Authenticated as whom, exactly? Does the link expire? Can it be forwarded and opened by someone who was never the intended recipient? And if that recipient is external per gap #2, can they even authenticate, or does large-file delivery just quietly not work for them?

None of this is spelled out, and it's precisely the largest reports, the ones with the most data in them, that hit this path. I'd build it as a signed, time-limited, single-use link tied to the original recipient, with every successful download logged. That logging also feeds directly into the monitoring section below.

4. What actually happens when a report is bigger than the platform can handle?

There's a Figma note that CSV exports cap at 500,000 records. The spec doesn't say what happens past that point: hard error, quiet truncation, or a partial file that looks complete but isn't.

Picture the real scenario: a compliance team pulls a full vulnerability export to hand to an auditor, and the file is missing 40% of the findings with nothing on the page saying so. They wouldn't find out until it caused a problem. These reports are explicitly meant to be evidence, so "silent" is the one behavior that can't be on the table. Whatever the cap ends up being, the file itself has to say so (a banner row in the CSV, a note on the PDF's cover page), and ideally the creation flow warns before generation if the selected scope looks likely to exceed it.

5. Scheduled runs can fail, and nothing says what happens then

The trigger/action mechanics are clear: daily, weekly, monthly, quarterly, generate, email. What's missing is the unhappy path. A data source times out, a dependency breaks, the run just fails. Retry? Notify someone? Does the Reports page even show that it happened?

This one matters because of who's on the other end of it. A CISO or an external auditor waiting on a scheduled report has no way to tell "nothing changed this cycle" apart from "it broke and nobody

noticed," and that gap tends to surface days or weeks later, usually when someone asks where the report is. Worth defining now: a retry policy, a failure notification to the report owner, and a visible failed-run state on the Reports page. This also sets up most of the alerting section in Part C.

A few more, worth a mention

- **Sync-board edge cases.** What happens if the source board is deleted, or loses a widget the report depends on, between one generation and the next?
- **Naming.** The spec text says "Summary Report," the Figma files say "Executive Report (PDF)." Small, but worth resolving before it lands in customer-facing copy.
- **Audit log scope.** The spec's audit-logging bullet only lists create/update/delete on the report object; generation and delivery aren't mentioned either way. That's exactly the record you'd want if gap #1 or #2 ever became a real incident, so it's worth confirming explicitly rather than assuming it's covered.
- **Empty results.** If a scope filter matches nothing, does the platform still generate and send an empty file, and does it say "0 results" clearly rather than just looking broken?
- **Scheduling load.** Reports will probably cluster around common times, daily at 6am, Monday mornings, and nothing in the spec addresses whether the generation pipeline can absorb that.
- **Saved-View scope: snapshot or live?** A report can use an existing saved View as its scope. If that View's filters change after the report is created, does the report pick up the change on its next generation, or does it keep whatever the View looked like at creation time? The spec doesn't say. It's the same snapshot-vs-dynamic question as "sync board changes," just for a different scope-selection path, and it deserves the same explicit answer.
- **What does "download" actually do?** The spec says customers can "download any report directly" from the Reports page. Does that regenerate fresh data on the spot, or serve the last generated file? Those are two different guarantees for something meant to be current, and the spec doesn't say which.

Part B: Test Plan

Priority is about what blocks release: **Critical** = release-blocking if broken, **High** = should be fixed first but a narrow workaround might exist, **Medium** = tracked but not necessarily blocking, **Low** = worth checking, not release-blocking.

A handful of rows are marked *(pending gap #N)*: these test whichever decision eventually gets made on an open question from Part A, rather than an already-specified behavior. I've written them now so the test plan is ready the moment those calls get made, instead of starting from scratch later.

B.1: Report creation (core flow)

ID	Description	Expected result	Priority
TC-01	Create a Summary Report from a Predefined Dashboard	Saves and shows up on the Reports page with the right type/module	Critical
TC-02	Create a Summary Report from a Custom Dashboard, sync off	Later edits to the source dashboard don't change future generations	Critical
TC-03	Same, sync on	Editing the source dashboard (add/remove a widget) changes the next generation to match	High
TC-04	Create an Operational Data report for a module with specific columns picked and reordered	Generated CSV has exactly those columns, in that order	Critical
TC-05	Try to save/generate with required fields missing (no name, no report type)	Blocked with a clear inline message, nothing half-saved	High
TC-06	Use an existing saved View as scope, then edit the View's filters *(pending decision: saved-view sync semantics)*	Report reflects whichever sync behavior gets decided, snapshot at creation or live at each generation; not specified yet	Medium
TC-07	Create a report set to "Only for me"	No other user in the org can see or list it	Critical (security)
TC-08	Create a report set to "Organization level"	Other org users with report-viewing permission can see and download it	Critical

B.2: "Save as Custom Report" entry points

ID	Description	Expected result	Priority
TC-09	From a findings list view, filter, then "Save as custom report"	Creation flow opens pre-filled with the active module/filters; only name/visibility left to fill in	High
TC-10	From a board view, "Save as custom report"	Pre-fills the board selection and current scope	High
TC-11	Spot-check the entry point on every module list/board view the spec says it should be on	Present and working everywhere it's supposed to be	Medium

B.3: On-demand generation & download

ID	Description	Expected result	Priority
TC-12	Generate & download a Summary Report immediately	PDF has correct dashboard content, trend data, and the configured time range applied	Critical
TC-13	Generate & download an Operational Data report immediately	CSV row count matches the live findings count for that scope	Critical
TC-14	Generate a report whose scope matches zero findings *(pending gap: empty results)*	File generated, "0 results" clearly communicated, not a blank/broken-looking export	High
TC-15	Generate an Operational Data report whose result set exceeds 500,000 rows *(pending gap #4)*	Visible truncation indicator present, no silent data loss	Critical (data integrity)

B.4: Reports page

ID	Description	Expected result	Priority
TC-16	Search/filter by name, module, creation date, creator, individually and combined	Results match the filters correctly	Medium
TC-17	Check a report's schedule status	Reports with an active workflow show it; ones without show a "create schedule" shortcut	Medium
TC-18	Download a report directly from the Reports page *(pending decision: regenerate vs. serve last artifact)*	Behavior matches whichever gets decided; the spec only says "download any report directly," not which	High
TC-19	Edit a saved report's scope, then regenerate	Change is saved and reflected in the next generation	High
TC-20	Delete a report with an active schedule *(pending gap: delete-with-schedule behavior)*	Either blocked with a warning, or deletes both the report and its schedule with explicit confirmation	High (destructive-action safety)

B.5: Scheduled delivery (Workflows)

ID	Description	Expected result	Priority
TC-21	Create a Daily/Weekly/Monthly/Quarterly workflow that generates and emails a saved report	Report is generated and emailed at the configured time	Critical
TC-22	Scheduled report under the destination's size limit	Delivered as a direct email attachment	High
TC-23	Scheduled report over the destination's size limit	A download link is sent instead; requires authentication before it works	Critical
TC-24	Configure a workflow recipient with an external, non-platform email address *(pending gap #2)*	Behavior matches the final policy decided for Gap #2; once defined, verify the restriction, any permission requirement, and the audit entry exactly	Critical (security)
TC-25	Simulate a scheduled generation failure, e.g. a broken data source *(pending gap #5)*	Not a testable pass/fail yet: depends entirely on the retry/notification policy decided for gap #5. Once defined (e.g. "retry twice, then mark the run Failed and notify the owner"), verify each of those steps exactly	Critical
TC-26	Disable a workflow	No further scheduled emails go out; Reports page reflects the disabled state	Medium

B.6: RBAC & audit

ID	Description	Expected result	Priority
TC-27	A user without report-creation permission tries to create one, via UI and API	Blocked both ways, with a clear permission error	Critical (security)
TC-28	A user without access to a specific module creates/generates a report for it *(pending gap #1)*	Behavior matches the final policy decided for Gap #1; once defined, verify the exact block/scope behavior	Critical
TC-29	Create, edit, delete a report object	Each action produces a correctly-attributed audit entry (actor, timestamp, action, report ID)	High
TC-30	Confirm whether generation/delivery events are captured in the audit log	Not specified in the PRD (only create/update/delete are listed); treat as an open gap until confirmed one way or the other, not a pass/fail check	Medium (documentation)

What I'd leave out of this round, and why

- **Load testing many concurrent scheduled generations.** Needs a dedicated perf environment with realistic data volume. Worth tracking as a follow-up, not a blocker for functional sign-off, though the underlying feasibility question is flagged in Part A.
- **Pixel-level visual regression on PDF rendering** across every widget type. This round is a functional smoke check, does it render, with the right data. Full visual QA is a follow-up once widget types settle down.
- **Delivery channels beyond email.** The release scope only commits to email, so Slack/ticketing integrations aren't tested until they're actually in scope.
- **Third-party email deliverability** (spam filtering, bounces). Assumed covered by the platform's existing email infrastructure, not specific to this feature.
- **Cross-browser/device testing of the creation flow.** Assumed covered by the platform's general UI test suite rather than re-tested per feature here.

Part C: BI & Monitoring Definition

Adoption / usage

Metric	What it measures	Why it matters	How to calculate it
Reports created, by type & module	Volume of report objects created, split Summary(PDF)/Operational(CSV) and by module	Shows which type/module actually resonates and informs roadmap priority	Count of `report.created` events, grouped by type and module, per period
% of orgs with ≥1 report	Breadth of adoption for a brand-new feature	The basic "is this landing" question	Distinct orgs with ≥1 report ÷ total active orgs
On-demand vs. scheduled mix	Share of actual report *generations* that come from a schedule vs. a manual click	The whole value prop here is automation; a low scheduled share means the scheduling UX needs work, not that the export itself is unused	Scheduled generation events ÷ all generation events, using the `trigger_type` field on the generation event log below. Counting report *objects* here would be misleading, since one scheduled report can generate hundreds of times a year while many on-demand reports generate once
"Save as Custom Report" usage	How often reports start from an existing view vs. the standalone flow	Tests the spec's own bet that starting from a familiar view lowers the barrier to creating a report	Creation events tagged `source: save_as` ÷ all creation events
Reports per active org (median / p90)	Depth of use, not just breadth	Tells "tried it once" orgs apart from power users, and power-user patterns often predict expansion conversations	Reports ÷ active orgs, median and p90

Quality / reliability

Metric	What it measures	Why it matters	How to calculate it
Generation success rate	% of generation attempts (on-demand + scheduled) that complete without error	This is the core promise: push a button, get a report	Successful generations ÷ total attempts, daily
Generation latency (p50 / p95)	Time from request to file ready	Slow generation kills trust in "generate now," and a rising trend catches scaling problems early	end_time minus start_time, by percentile and report type
Delivery success rate	% of scheduled delivery attempts that go out successfully	A generated-but-undelivered report is invisible to the customer, with the same trust cost as a failed generation	Delivery attempts accepted by the sending provider ÷ total attempts. If bounce/delivery telemetry is available, tighten this to attempts confirmed delivered to the recipient's mailbox rather than just accepted for sending, since "accepted" and "arrived" aren't the same guarantee
Large-file link-fallback rate	% of deliveries falling back to a link instead of an attachment	If this is high, the size threshold is probably wrong for real customer data: a product conversation, not just an ops one	Link-fallback deliveries ÷ total scheduled deliveries
Truncation rate	% of Operational Data generations hitting the 500K-row cap	Same idea as above, for the CSV cap; also tells us whether the truncation warning from Part A is actually getting used	Generations flagged `truncated: true` ÷ total CSV generations
Scheduled-run on-time rate	% of scheduled generations firing within an acceptable window of their configured time	This is the reliability promise behind "automated delivery, at the right time"	Runs starting within ~5 min of schedule ÷ total scheduled runs

What pages someone vs. what waits for the weekly review

Page on-call immediately when:

- Generation success rate drops below ~95% over a rolling 30 minutes (or an absolute count for quiet periods, say 5 failures in a row). That's a systemic break, not one bad report.

- Delivery success rate drops below ~95% over the same kind of window. Customers are actively not getting reports they're expecting.
- Scheduled-run backlog passes a threshold (e.g. 50+ runs delayed more than 15 minutes), catching a queue problem before it snowballs.
- Generation latency p95 breaches an SLO (say, 2 minutes for CSVs, 5 for PDFs) for more than a few minutes straight.

These are proposed starting thresholds, not numbers validated against real production traffic. I'd expect to tune all of them after launch based on actual baseline volume, customer expectations, and whatever SLOs get formally agreed.

Fine to check weekly, not paged:

- Adoption trends: reports created, % orgs onboarded, on-demand/scheduled split. Roadmap input, not an incident.
- Truncation and link-fallback rate trends: a product conversation about thresholds, not an emergency.
- "Save as Custom Report" usage: a UX signal.
- Reports-per-org distribution: an account-health question.

Data that doesn't exist yet and probably should

- **A generation event log.** One entry per attempt: report ID, trigger type, start/end time, success or failure and why, whether it was truncated, output size or row count. The PRD only specifies audit logging for create/update/delete; generation-level logging is not defined.
- **Delivery events.** Per scheduled send: recipients, delivery method (attachment vs. link), size, outcome, and, for link-based delivery, the actual download events after the fact. This feeds the reliability metrics above and is also the record that closes gaps #2 and #3: knowing who really accessed a report, not just who it was addressed to.
- **Manual downloads from the Reports page.** Not specified in the PRD whether these are tracked at all. If they aren't, "reports per org" would undercount actual usage.
- **A simple "first report created" timestamp per org.** Small thing, but it turns the adoption metric into a lookup instead of something re-derived from raw events every time someone asks.